

JavaScript 导论

- 本课程主要介绍 JavaScript 核心语法和浏览器网页开发的基本知识，主要可以分成以下四大部分：
 - 基本语法
 - 标准库
 - 浏览器 API
 - DOM

JavaScript在HTML中的引用方式

- 在<head></head>标签内编写JavaScript

```
1  <!DOCTYPE html>
2  <html xmlns="http://www.w3.org/1999/xhtml">
3  <head>
4      <title></title>
5      <script type="text/javascript">
6          //这里编写JavaScript程序
7      </script>
8  </head>
9  <body>
10 </body>
11 </html>
```

- 在<body></body>标签内编写JavaScript

```
1  <!DOCTYPE html>
2  <html xmlns="http://www.w3.org/1999/xhtml">
3  <head>
4      <title></title>
5  </head>
6  <body>
7      <script type="text/javascript">
8          //这里编写JavaScript程序
9      </script>
10 </body>
11 </html>
```

JavaScript在HTML中的引用方式

- 在元素事件中引入，在元素的某一个属性中直接编写JavaScript程序或调用JavaScript函数，这个属性指的是元素的“事件属性”

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5 </head>
6 <body>
7   <input type="button" onClick="alert('绿叶学习网')" value="按钮"/>
8 </body>
9 </html>
```

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <script type="text/javascript">
6     function alertMessage()
7     {
8       alert("绿叶学习网");
9     }
10  </script>
11 </head>
12 <body>
13   <input type="button" onclick="alertMessage()" value="按钮"/>
14 </body>
15 </html>
```

JavaScript在HTML中的引用方式

- 引入外部文件，把JavaScript程序存放在一个后缀名为.js的文件中，然后使用script标签来引用。script标签可以放在head标签内，也可以放在body标签中
- 语法：

```
1 | <script src="scripts/main.js"></script>
```

- 如果脚本文件使用了非英语字符，应该注明字符的编码，所加载的脚本必须是纯 JavaScript 代码，不能有HTML代码和<script>标签

```
<script charset="utf-8" src="https://www.example.com/script.js"></script>
```

JavaScript在HTML中的引用方式

- 为了防止攻击者篡改外部脚本，script标签允许设置一个integrity属性，写入该外部脚本的Hash 签名，用来验证脚本的一致性

```
<script src="/assets/application.js"  
  integrity="sha256-TvVUHfSfftWg1rcfL6TIJ0XKEGrgLyEq6lEpcmrG9qs=">  
</script>
```

- script 的 integrity 属性指定了外部脚本 /assets/application.js 的 SHA256 签名，一旦有人改了脚本，导致 SHA256 签名不匹配，浏览器就会拒绝加载

浏览器加载JavaScript的过程

- 浏览器加载 JavaScript 脚本，主要通过<script>元素完成,正常的网页加载流程如下：
 - 浏览器一边下载 HTML 网页，一边开始解析，不等下载到完，就开始解析
 - 解析过程中，浏览器发现<script>元素，就暂停解析，把网页渲染的控制权转交给 JavaScript 引擎
 - 如果<script>元素引用了外部脚本，就下载该脚本再执行，否则就直接执行代码
 - JavaScript 引擎执行完毕，控制权交还渲染引擎，恢复往下解析 HTML 网页。
- 加载外部脚本时，浏览器会暂停页面渲染，等待脚本下载并执行完成后，再继续渲染。原因是 JavaScript 代码可以修改 DOM，所以必须把控制权让给它，否则会导致复杂的线程竞赛的问题。

浏览器加载JavaScript的过程

- 如果外部脚本加载时间很长（一直无法完成下载），那么浏览器就会一直等待脚本下载完成，造成网页长时间失去响应，浏览器就会呈现“假死”状态，这被称为“阻塞效应”
- 为了避免这种情况，较好的做法是将<script>标签都放在页面底部，而不是头部。这样即使遇到脚本失去响应，网页主体的渲染也已经完成了，用户至少可以看到内容，而不是面对一张空白的页面。
- 如果某些脚本代码非常重要，一定要放在页面头部的话，最好直接将代码写入页面，而不是连接外部脚本文件，这样能缩短加载时间

浏览器加载JavaScript的过程

- 脚本文件都放在网页尾部加载，还有一个好处。因为在 DOM 结构生成之前就调用 DOM 节点，JavaScript 会报错，如果脚本都在网页尾部加载，就不存在这个问题，因为这时 DOM 肯定已经生成了

```
<head>
  <script>
    console.log(document.body.innerHTML);
  </script>
</head>
<body>
</body>
```

上面代码执行时会报错，因为此时 document.body 元素还未生成

浏览器加载JavaScript的过程

- 如果有多个script标签，比如下面这样

```
<script src="a.js"></script>  
<script src="b.js"></script>
```

- 浏览器会同时并行下载a.js和b.js，但是，执行时会保证先执行a.js，然后再执行b.js，即使后者先下载完成，也是如此
- 脚本的执行顺序由它们在页面中的出现顺序决定，这是为了保证脚本之间的依赖关系不受到破坏

从编译原理的角度学习基本语法

- 编译原理之词法分析、语法分析、语义分析
 - 词法分析 (Lexical analysis) 是编译过程的第一个阶段，任务是从左到右一个字符一个字符地读入源程序，即对构成源程序的字符流进行扫描然后根据构词规则**识别单词**(也称单词符号或符号)
 - 语法分析 (Syntax analysis或Parsing) 是编译过程的一个逻辑阶段，任务是在词法分析的基础上**将单词序列组合成各类语法短语**，如函数、语句、表达式等等，形成语法树
 - 语义分析 (Syntax analysis) 是编译过程的一个逻辑阶段，任务是对结构上正确的源程序进行上下文有关性质的审查，**进行类型审查**

理解JavaScript的编译过程

- 对于常见编译型语言（例如：Java）来说，编译步骤分为：词法分析->语法分析->语义分析->代码优化和字节生成
- 对于解释型语言（例如JavaScript）来说，通过词法分析和语法分析得到语法树后，就可以开始解释执行
 - 词法分析将字符流(char stream)转换为记号流(token stream)，每个记号代表了源代码中的一个基本语法单元，比如关键字、标识符、运算符、常量和分隔符等

理解JavaScript的编译过程

- JavaScript编译过程

- 词法分析将字符流(char stream) 按照ECMAScript标准转换为记号流(token stream)

代码：var result = testNum1 - testNum2;

词法分析后：

1. NAME "result"
2. EQUALS
3. NAME "testNum1"
4. MINUS
5. NAME "testNum2"
6. SEMICOLON

理解JavaScript的编译过程

- JavaScript编译过程

- 语法分析得到语法树，当JavaScript解释器在构造语法树的时候，如果发现无法构造，就会报语法错误，并结束整个代码块的解析

条件语句

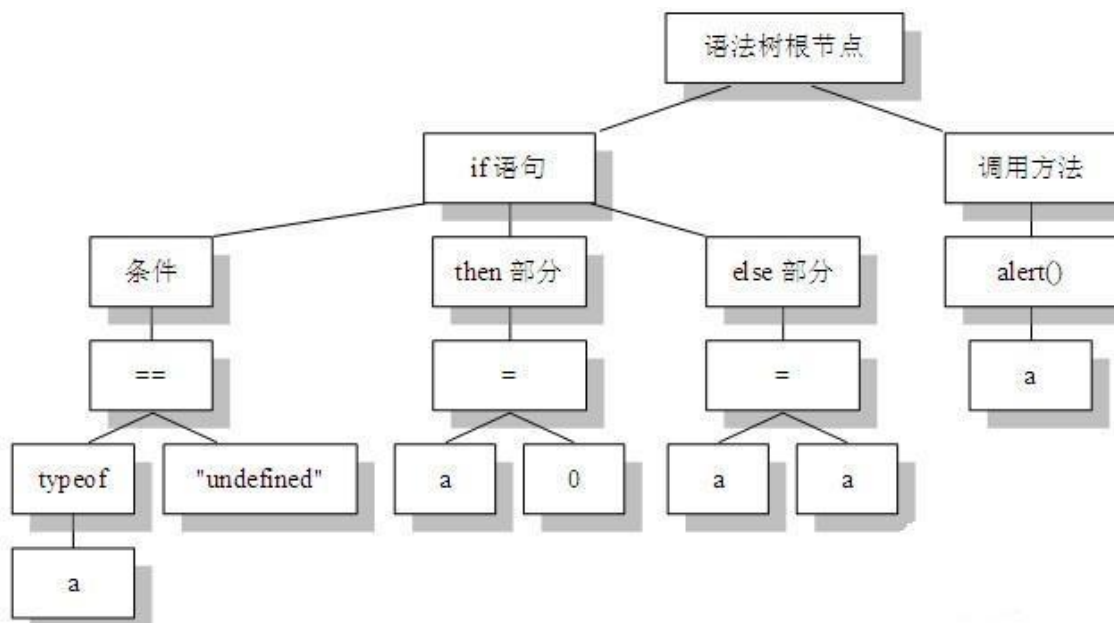
```
if(typeof a == "undefined" )
```

```
    { a = 0; }
```

```
else
```

```
    { a = a; }
```

```
alert(a);
```



理解JavaScript的编译过程

- JavaScript编译过程

- 当JavaScript引擎解析脚本时，它会在预编译期对所有声明的变量和函数进行处理，并且是先预声明变量（把所有var变量创建，默认值为undefined），再预定义函数（把定义式的函数优先执行）

- 进入解释执行阶段

JavaScript引擎

- JavaScript 引擎的主要作用是，**读取网页中的 JavaScript 代码，对其处理后运行**
- JavaScript 是一种解释型语言，也就是说，它不需要编译，由解释器实时运行。这样的好处是运行和修改都比较方便，刷新页面就可以重新解释；缺点是每次运行都要调用解释器，系统开销较大，运行速度慢于编译型语言
- 为了提高运行速度，目前的浏览器都将 JavaScript 进行**一定程度的编译**，生成类似字节码 (bytecode) 的**中间代码**，以提高运行速度

JavaScript引擎

- 早期浏览器内部对 JavaScript 的处理过程如下：
 - 读取代码，进行词法分析（Lexical analysis），将代码分解成词元（token）
 - 对词元进行语法分析（parsing），将代码整理成“语法树”（syntax tree）
 - 使用“翻译器”（translator），将代码转为字节码（bytecode）
 - 使用“字节码解释器”（bytecode interpreter），将字节码转为机器码

JavaScript引擎

- 逐行解释将字节码转为机器码，是很低效的。为了提高运行速度，现代浏览器改为采用“即时编译”（**Just In Time compiler, 缩写 JIT**）。在使用 JIT 编译器的语言中，源代码会首先被解释器解释执行，而当某些部分的代码被**频繁执行时**，JIT 编译器会将这些部分的代码编译成机器码，从而提高执行速度。

在编程中，通常有两种翻译为机器语言的方式。我们可以使用解释器（interpreter）或者编译器（compiler）。

在使用解释器时，这种翻译几乎是一行一行实时进行的。

编译器使用另一种方式，它会提前翻译好并且记录下来，而不是动态进行翻译。

JIT 是编译器（Compiler）和解释器（Interpreter）的融合或者组合

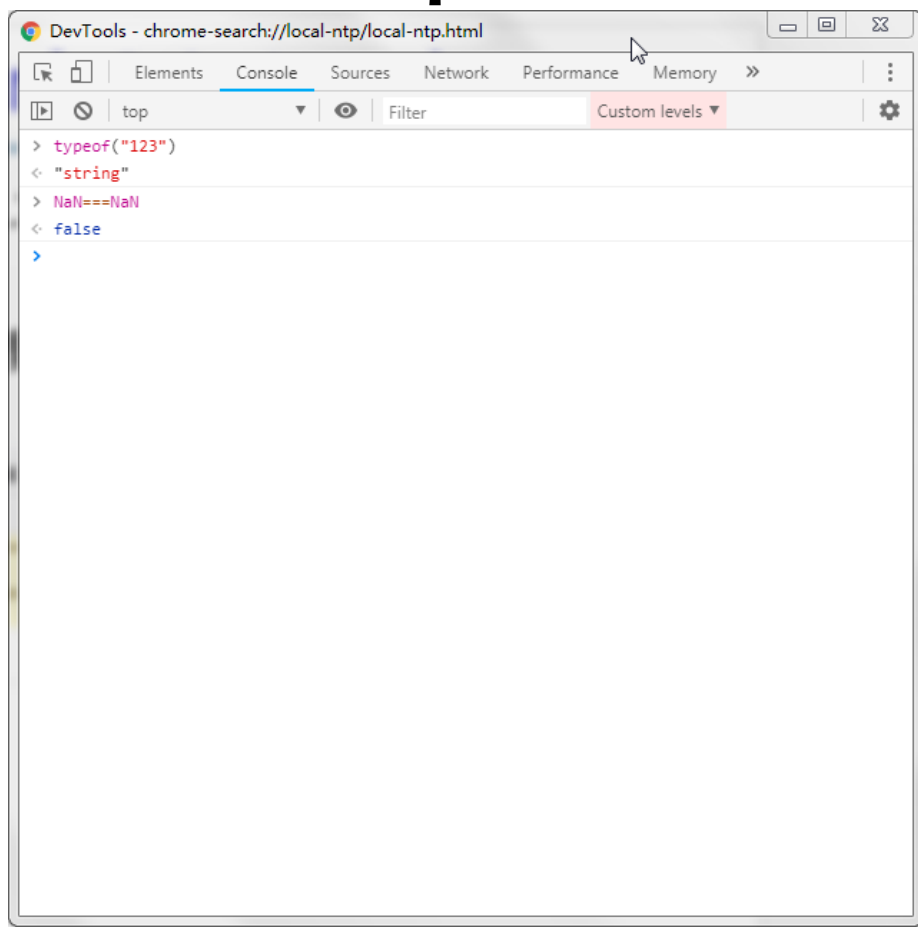
JavaScript引擎

- 字节码不能直接运行，而是运行在一个虚拟机（Virtual Machine）之上，**一般也把虚拟机称为 JavaScript 引擎**。并非所有的 JavaScript 虚拟机运行时都有字节码，**有的 JavaScript 虚拟机基于源码**，即只要有可能，就通过 JIT（just in time）编译器直接把源码编译成机器码运行，省略字节码步骤。这一点与其他采用虚拟机（比如 Java）的语言不尽相同。这样做的目的，是为了尽可能地优化代码、提高性能，但也存在问题。
- 下面是目前最常见的一些 JavaScript 虚拟机：
 - Chakra (Microsoft Internet Explorer)
 - Nitro/JavaScript Core (Safari)
 - Carakan (Opera)
 - SpiderMonkey (Firefox)
 - V8 (Chrome, Chromium)
 - JavaScriptCore (Webkit)

<https://zhuanlan.zhihu.com/p/26669846>

JavaScript试验环境

- 浏览器的“开发者工具”（Developer Tools）里面的“控制台”（console），就是运行JavaScript代码的理想环境



进入控制台以后，就可以在提示符后输入代码，然后按Enter键，代码就会执行。如果按Shift + Enter键，就是代码换行，不会触发执行

参考在线资源

- JavaScript 教程，阮一峰，网道
 - <https://wangdoc.com/javascript/>
- JavaScript, MDN
 - <https://developer.mozilla.org/zh-CN/docs/Web/JavaScript>
- JavaScript入门教程，绿叶学习网
 - http://www.lvvestudy.com/les_js/js_list.aspx

主要内容

- 基本语法
- 标准库

备注： 本部分将专注于 JavaScript 语言本身，
而非局限于网页或其他宿主环境

语句和表达式

- JavaScript 程序的执行单位为行 (line)
- 语句 (statement) 指为了完成任务而进行的操作
 - 语句以分号结尾，一个分号就表示一个语句结束
 - 多个语句可以写在一行内
- 表达式 (expression) 指为了得到返回值的计算式

`var a = 1 + 3 ;`

语句 表达式

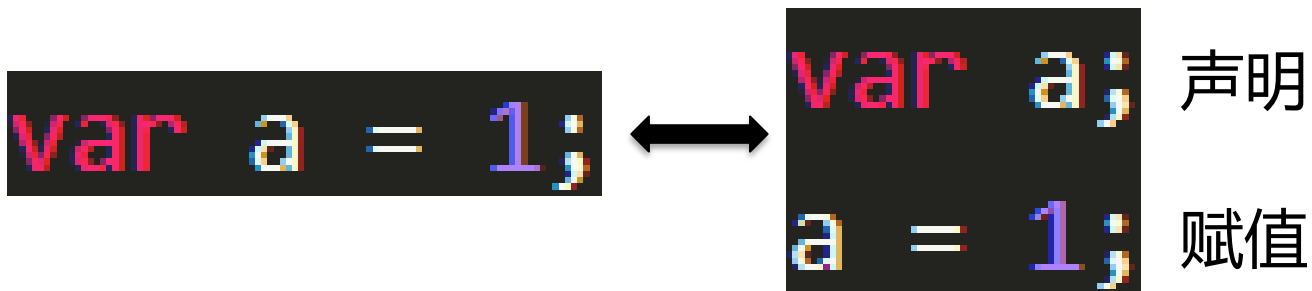
`;;;`

分号前面可以没有任何内容，JavaScript 引擎将其视为空语句

"表达式" (expression) 是一个单纯的运算过程，总是有返回值
"语句" (statement) 是执行某种操作，一般情况没有返回值

变量

- 变量是对“值”的具名引用。变量就是为“值”起名，然后引用这个名字，就等同于引用这个值。变量的名字就是变量名



先声明变量a，然后在变量a与数值1之间建立引用关系，称为将数值1“赋值”给变量a。以后，引用变量名a就会得到数值1。**最前面的var，是变量声明命令。**它表示通知解释引擎，要创建一个变量a

注意：JavaScript 的变量名区分大小写，A和a是两个不同的变量

变量

- 如果只是声明变量而没有赋值，则该变量的值是undefined，一个表示“无定义”的特殊值。如果一个变量没有声明就直接使用会报错，告诉你变量未定义

```
var a;  
a // undefined
```

```
x  
// ReferenceError: x is not defined
```

- JavaScript 是一种**动态类型**语言，变量的类型没有限制，变量可以随时更改类型

```
var a = 1;  
a = 'hello';
```

变量的作用域

- 变量的作用域

- 变量的作用域是指变量在程序中的有效范围
- 在ES5中，变量根据作用域可以分为2种：全局变量和局部变量

```
1  <!DOCTYPE html>
2  <html xmlns="http://www.w3.org/1999/xhtml">
3  <head>
4      <title></title>
5      <script type="text/javascript">
6          var a;    //该变量在函数外声明，作用于整个脚本代码；
7          function send()
8          {
9              a="绿叶学习网";
10             var b="JavaScript入门教程"; //该变量在函数体内声明，只作用于该函数体
11             alert(a+b);
12         }
13         send();
14     </script>
15 </head>
16 <body>
17 </body>
18 </html>
```

区块与作用域

- JavaScript 使用大括号将多个相关的语句组合在一起，称为“区块”（block），单独使用区块并不常见，区块往往用来构成其他更复杂的语法结构，比如for、if、while、function等
- 对于var命令来说，JavaScript 的区块不构成单独的作用域（scope）

```
function myFunction() {  
  if (true) {  
    var localVar = 10;  
    console.log(localVar); // 输出 10  
  }  
  console.log(localVar); // 仍然可以访问 localVar，在函数内部可见，输出 10  
}  
  
myFunction();  
console.log(localVar); // 报错，因为在函数外部无法访问 localVar
```

解释：在区块内部使用var命令声明并赋值了变量a，在区块外部变量a依然有效，区块对于var命令不构成单独的作用域，与不使用区块的情况没有任何区别

var, let 和 const

- var、let 和 const 都是 JavaScript 中用来声明变量的关键字，let 和 const 关键字是在 ES6 中新增的
- var 与 let 的区别 – 作用域区别
 - var 声明的变量的作用域是当前执行上下文，如果是在任何函数外面，则是全局执行上下文，如果在函数里面，则是当前函数执行上下文。var 声明的变量的作用域只能是全局或者整个函数块的
 - let 声明的变量的作用域则是它当前所处代码块，即它的作用域既可以是全局或者整个函数块，也可以是 if、while、switch 等用 {} 限定的代码区块
 - var 和 let 的作用域规则都是一样的，其声明的变量只在其声明的块或子块中可用

var, let 和 const

- var、let 和 const 都是 JavaScript 中用来声明变量的关键字，let 和 const 关键字是在 ES6 中新增的
- var 与 let 的区别 – 作用域区别

```
function varTest() {  
  var a = 1;  
  
  {  
    var a = 2; // 函数块中, 同一个变量  
    console.log(a); // 2  
  }  
  
  console.log(a); // 2  
}  
  
function letTest() {  
  let a = 1;  
  
  {  
    let a = 2; // 代码块中, 新的变量  
    console.log(a); // 2  
  }  
  
  console.log(a); // 1  
}
```

var 声明的变量的作用域只能是全局或者整个函数块的，产生变量提升现象

let 声明的变量的作用域则是它当前所处代码块，区块可以构成独立作用域

var, let和const

- var与let的区别

- var 允许在同一作用域中重复声明，而 let 不允许在同一作用域中重复声明，否则将抛出异常

var 相关示例代码:

```
var a = 1;
var a = 2;

console.log(a) // 2

function test() {
  var a = 3;
  var a = 4;
  console.log(a) // 4
}

test()
```

let 相关示例代码:

```
if(false) {
  let a = 1;
  let a = 2; // SyntaxError: Identifier 'a' has already been declared
}

switch(index) {
  case 0:
    let a = 1;
    break;

  default:
    let a = 2; // SyntaxError: Identifier 'a' has already been declared
    break;
}
```

不用等index算出具体值就会做重复性检查

全局作用域和函数作用域
中的变量提升

let 声明的重复性检查是发生在词法分析阶段，也就是在代码正式开始执行之前就会进行检查

var, let 和 const

- let 与 const 异同

- const 与 let 很类似，都具有上面提到的 let 的特性，唯一区别就在于 const 声明的是一个只读变量，声明之后不允许改变其值。因此，const 一旦声明必须初始化，否则会报错

示例代码:

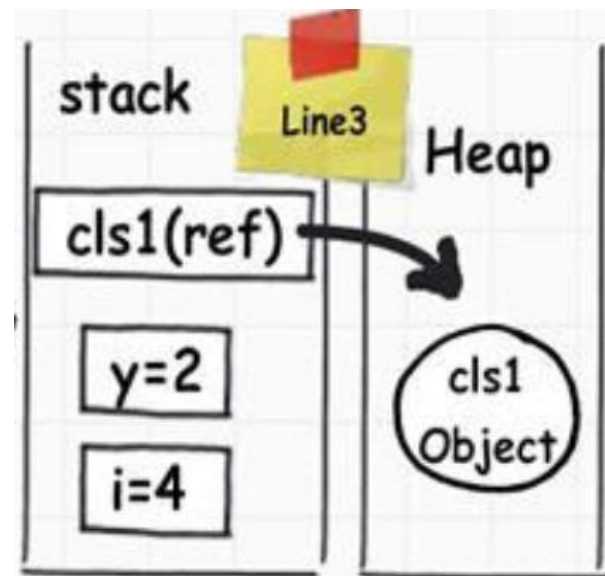
```
let a;  
const b = "constant"  
  
a = "variable"  
b = 'change' // TypeError: Assignment to constant variable
```

var, let和const

- 如何理解const声明之后不允许改变其值？
 - 其实 const 保证的不是变量的值不变，而是保证变量指向的内存地址所保存的数据不允许改动（即栈内存存在的值和地址）
 - JavaScript 的数据类型分为两类：原始值类型和对象（Object类型）

对于原始值类型（undefined、null、true/false、number、string），值就保存在变量指向的那个内存地址（在栈中），因此 const 声明的原始值类型变量等同于常量

对于对象类型（object，array，function等），变量指向的内存地址其实是保存了一个指向实际数据的指针，所以 const 只能保证指针是不可修改的，至于指针指向的数据结构是无法保证其不能被修改的（在堆中）



var, let 和 const

- 如何理解const声明之后不允许改变其值?
 - 其实 const 保证的不是变量的值不变，而是保证变量指向的内存地址所保存的数据不允许改动（即栈内存存在的值和地址）

示例代码:

```
const obj = {  
  value: 1  
}  
  
obj.value = 2  
  
console.log(obj) // { value: 2 }  
  
obj = {} // TypeError: Assignment to constant variable
```

对于对象类型的值，const 只能保证指针是不可修改的，至于指针指向的数据结构是无法保证其不能被修改的

变量提升

- JavaScript 引擎的工作方式是，先解析代码，获取所有被声明的变量，然后再一行一行地运行。结果就是所有的变量声明语句，都会被提升到代码的头部，这就叫**变量提升 (hoisting)**

```
console.log(a);  
var a = 1;
```

=

```
var a;  
console.log(a);  
a = 1;
```

问题：在控制台输出的是什么？是1吗？
答案：undefined，不是报错

解释：首先使用console.log方法，在控制台（console）显示变量a的值。这时变量a还没有声明和赋值，所以这是一种错误做法，但是实际上因为存在变量提升不会报错



当心坑洞

标识符

- 标识符 (identifier) 指的是用来识别各种值的合法名称，常见的标识符有变量名、函数名等
- JavaScript 语言的标识符对大小写敏感，标识符有命名规则，不符合规则的就是非法标识符，JavaScript 引擎遇到非法标识符会报错
 - 第一个字符可以是任意 Unicode 字母（包括英文字母和其他语言的字母）以及美元符号 (\$) 和下划线 (_)
 - 第二个字符及后面的字符，除了 Unicode 字母、美元符号和下划线，还可以用数字 0-9

标识符

合法标识符

arg0

_tmp

\$elem

π

var 临时变量 = 1;

中文是合法的标识符
可以用作变量名

非法标识符

1a // 第一个字符不能是数字

23 // 同上

*** // 标识符不能包含星号

a+b // 标识符不能包含加号

-d // 标识符不能包含减号或连词线

JavaScript 有一些保留字，不能用作标识符：arguments、break、case、catch、class、const、continue、debugger、default、delete、do、else、enum、eval、export、extends、false、finally、for、function、if、implements、import、in、instanceof、interface、let、new、null、package、private、protected、public、return、static、super、switch、this、throw、true、try、typeof、var、void、while、with、yield。

保留字在JavaScript语言中有特定含义，是JavaScript语法中一部分

注释

- JavaScript 提供两种注释的写法：
 - 单行注释，用//起头
 - 多行注释，放在/*和*/之间
- `<!-- -->` 如果在此注释符号内编写 JavaScript 脚本，对于不支持 JavaScript 的浏览器，将会把编写的 JavaScript 脚本作为HTML注释处理。

```
<script>  
<!--  
document.write("您的浏览器支持JavaScript脚本!");  
//-->  
</script>
```



当心坑洞

注释

- **HTML注释**
 - `<!--注释的内容-->`
- **CSS注释**
 - `/*注释的内容*/`
- **JavaScript 提供两种注释的写法：**
 - 单行注释，用`//`起头
 - 多行注释，放在`/*`和`*/`之间
 - `<!--注释的内容-->`分情况讨论

条件语句

- if 结构

```
if (m === 3) {  
    m += 1;  
}
```

注意: if后面的表达式之中，可以有严格相等运算符（===）和相等运算符（==），它们的区别是相等运算符（==）比较两个值是否相等，严格相等运算符（===）比较它们是否为“同一个值”。如果两个值不是同一类型，严格相等运算符（===）直接返回false，而相等运算符（==）会将它们转换成同一个类型，再用严格相等运算符进行比较

<https://wangdoc.com/javascript/operators/comparison.html>

```
1 === "1" // false  
true === "true" // false
```

条件语句

- if 结构

```
var x = 1;  
var y = 2;  
if (x = y) {  
    console.log(x);  
}  
// "2"
```

```
if (x = 2) { // 不报错  
if (2 = x) { // 报错
```

小技巧

注意: 不要把严格相等运算符 (===) 和相等运算符 (==) 写成混淆赋值表达式 (=)，为了避免这种情况，有些开发者习惯将常量写在运算符的左边，这样一旦不小心将相等运算符写成赋值运算符，就会报错，因为常量不能被赋值

条件语句

- if...else 结构

```
if (m === 3) {  
    // 满足条件时，执行的语句  
} else {  
    // 不满足条件时，执行的语句  
}
```

```
if (m === 0) {  
    // ...  
} else if (m === 1) {  
    // ...  
} else if (m === 2) {  
    // ...  
} else {  
    // ...  
}
```

对同一个变量进行多次判断时，多个 if...else 语句可以连写在一起

条件语句

- **if...else 结构**

- **else代码块总是与离自己最近的那个if语句配对**

```
var m = 1;
var n = 2;

if (m !== 1)
if (n === 2) console.log('hello');
else console.log('world');
```



```
if (m !== 1) {
  if (n === 2) {
    console.log('hello');
  } else {
    console.log('world');
  }
}
```

```
if (m !== 1) {
  if (n === 2) {
    console.log('hello');
  }
} else {
  console.log('world');
}
// world
```

上面代码不会有任何输出，else代码块不会得到执行，因为它跟着的是最近的那个if语句

条件语句

- switch 结构

```
switch (fruit) {  
  case "banana":  
    // ...  
    break;  
  case "apple":  
    // ...  
    break;  
  default:  
    // ...  
}
```

根据变量fruit的值，选择执行相应的case。如果所有case都不符合，则执行最后的default部分

```
var x = 1;  
  
switch (x) {  
  case 1:  
    console.log('x 等于1');  
  case 2:  
    console.log('x 等于2');  
  default:  
    console.log('x 等于其他值');  
}  
// x等于1  
// x等于2  
// x等于其他值
```

每个case代码块内部的break语句不能少，否则会接下去执行下一个case代码块，而不是跳出switch结构

条件语句

- switch 结构

— 需要注意的是，switch语句后面的表达式，与case语句后面的表示式比较运行结果时，采用的是**严格相等运算符（===）**，而不是相等运算符（==），这意味着比较时不会发生类型转换



当心坑洞

```
var x = 1;

switch (x) {
  case true:
    console.log('x 发生类型转换');
    break;
  default:
    console.log('x 没有发生类型转换');
}

// x 没有发生类型转换
```

```
> var x = 1;
< undefined
> typeof x
< "number"
> typeof true
< "boolean"
> x == true
< true
> x === true
< false
>
```

条件语句

- 三元运算符

- 用于逻辑判断，如果“条件”为true，则返回“表达式1”的值，否则返回“表达式2”的值

(条件) ? 表达式1 : 表达式2

```
var even = (n % 2 === 0) ? true : false;
```

=

```
var even;  
if (n % 2 === 0) {  
    even = true;  
} else {  
    even = false;  
}
```

```
var msg = '数字' + n + '是' + (n % 2 === 0 ? '偶数' : '奇数');
```

循环语句

• while 循环

```
var i = 0;

while (i < 100) {
  console.log('i 当前为: ' + i);
  i = i + 1;
}
```

• for 循环

```
for (初始化表达式; 条件; 递增表达式)
  语句
```

// 或者

```
for (初始化表达式; 条件; 递增表达式) {
  语句
}
```

- 初始化表达式 (initialize) : 确定循环变量的初始值, 只在循环开始时执行一次
- 条件表达式 (test) : 每轮循环开始时, 都要执行这个条件表达式, 只有值为真, 才继续进行循环
- 递增表达式 (increment) : 每轮循环的最后一个操作, 通常用来递增循环变量

循环语句

- do...while 循环

- 不管条件是否为真，do...while循环至少运行一次，while语句后面的分号注意不要省略

```
do  
    语句  
while (条件);  
  
// 或者  
do {  
    语句  
} while (条件);
```

循环语句

- **break 语句和 continue 语句**

- break语句和continue语句都具有跳转作用，可以让代码不按既有的顺序执行
- break语句用于跳出循环，continue语句用于终止本轮循环，返回循环结构的头部，开始下一轮循环

```
for (var i = 0; i < 5; i++) {  
    console.log(i);  
    if (i === 3)  
        break;  
}  
// 0  
// 1  
// 2  
// 3
```

```
var i = 0;  
  
while (i < 100){  
    i++;  
    if (i % 2 === 0) continue;  
    console.log('i 当前为: ' + i);  
}
```

只有在i为奇数时，才会输出i的值
如果i为偶数，则直接进入下一轮循环

循环语句

- 标签 (label)

- JavaScript 语言允许，语句的前面有标签 (label) ，相当于定位符，用于跳转到程序的任意位置
- 标签可以是任意的标识符，但不能是保留字，语句部分可以是任意语句
- 标签通常与break语句和continue语句配合使用，跳出特定的循环

```
label:  
  语句
```

循环语句

- 标签 (label)

```
top:
  for (var i = 0; i < 3; i++){
    for (var j = 0; j < 3; j++){
      if (i === 1 && j === 1) break top;
      console.log('i=' + i + ', j=' + j);
    }
  }
// i=0, j=0
// i=0, j=1
// i=0, j=2
// i=1, j=0
```

双重循环区块，break命令后面加上了top标签（注意，top不用加引号），满足条件时，直接跳出双层循环。如果break语句后面不使用标签，则只能跳出内层循环，进入下一次的外层循环

循环语句

- 标签 (label)

```
foo: {  
  console.log(1);  
  break foo;  
  console.log('本行不会输出');  
}  
console.log(2);  
// 1  
// 2
```

标签也可以用于跳出代码块
示例中代码执行到 **break foo**, 就会跳出区块

循环语句

- 标签 (label)

```
top:
  for (var i = 0; i < 3; i++){
    for (var j = 0; j < 3; j++){
      if (i === 1 && j === 1) continue top;
      console.log('i=' + i + ', j=' + j);
    }
  }
```

```
// i=0, j=0
// i=0, j=1
// i=0, j=2
// i=1, j=0
// i=2, j=0
// i=2, j=1
// i=2, j=2
```

continue语句也可以与标签配合使用

示例中**continue**命令后面有一个标签名，满足条件时，会跳过当前循环，直接进入下一轮外层循环

如果**continue**语句后面不使用标签，则只能进入下一轮的内层循环，去掉**top**试一下！ (i=1, j=2)

主要内容

- 基本语法
- 数据类型
- 运算符
- 标准库

数据类型简介

- JavaScript 语言的每一个值，都属于某一种数据类型。JavaScript 的数据类型，共有六种：
 - 数值 (number)：整数和小数（比如1和3.14）
 - 字符串 (string)：文本（比如Hello World）
 - 布尔值 (boolean)：表示真伪的两个特殊值，即 true（真）和 false（假）
 - undefined：表示“未定义”或不存在，即由于目前还没有定义，所以此处暂时没有任何值
 - null：表示空值，即此处的值为空
 - 对象 (object)：各种值组成的集合

数据类型简介

- 数值、字符串、布尔值这三种类型，合称为**原子类型 (primitive type)** 的值，即它们是最基本的数据类型，不能再细分了
- 对象则称为**复合类型 (complex type)** 的值，因为一个对象往往是多个原子类型的值的合成，可以看作是一个存放各种值的容器
- undefined和null可以看成两个特殊值

数据类型简介

- 对象是最复杂的数据类型，又可以分成三个子类型
 - 狭义的对象 (object)
 - 数组 (array)
 - 函数 (function)
- 狭义的对象和数组是两种不同的数据组合方式，函数其实是处理数据的方法，JavaScript 把它当成一种数据类型，可以赋值给变量，这为编程带来了很大的灵活性，也为 JavaScript 的“函数式编程”奠定了基础

typeof运算符

- typeof运算符可以返回一个值的数据类型

```
typeof 123 // "number"  
typeof '123' // "string"  
typeof false // "boolean"
```

数值、字符串、布尔值分别返回
number、string、boolean

```
function f() {}  
typeof f  
// "function"
```

函数返回function，对象的子类型之一
(Brendan Eich觉得：函数太常用了，如果typeof返回"object"，开发者会区分函数和普通对象很困难。

```
typeof undefined  
// "undefined"
```

undefined返回undefined

```
typeof window // "object"  
typeof {} // "object"  
typeof [] // "object"
```

对象返回object
在JavaScript内部，数组本质上
只是一种特殊的对象

```
typeof null // "object"
```

null返回object

typeof运算符

- typeof运算符可以返回一个值的数据类型

```
typeof null // "object"
```

null返回object

注意： `null` 的类型是 `object`，这是由于历史原因造成的。1995年的JavaScript 语言第一版，只设计了五种数据类型（对象、整数、浮点数、字符串和布尔值），没考虑 `null`，只把它当作 `object` 的一种特殊值。后来 `null` 独立出来，作为一种单独的数据类型，为了兼容以前的代码，`typeof null` 返回 `object` 就没法改变了



当心坑洞

主要内容

- 基本语法
- 数据类型
 - null和undefined
 - 布尔值
 - 数值
 - 字符串
 - 对象
- 运算符
- 标准库

null和undefined

- null与undefined都可以表示“没有”，含义非常相似，在if语句中它们都会被自动转为false，相等运算符（==）甚至直接报告两者相等

```
if (!undefined) {  
  console.log('undefined is false');  
}  
// undefined is false  
  
if (!null) {  
  console.log('null is false');  
}  
// null is false  
  
undefined == null  
// true
```

既然含义与用法都差不多，为什么要同时设置两个这样的值，这不是无端增加复杂度，令初学者困扰吗？



null和undefined

- 1995年 JavaScript 诞生时，最初像 Java 一样，只设置了 null 表示“无”。根据 C 语言的传统，null 可以自动转为 0
- JavaScript 的设计者 Brendan Eich，觉得这样做还不够。首先，第一版的 JavaScript 里面，null 就像在 Java 里一样，被当成一个对象，Brendan Eich 觉得表示“无”的值最好不是对象。其次，那时的 JavaScript 不包括错误处理机制，Brendan Eich 觉得，如果 null 自动转为 0，很不容易发现错误
- 因此，他又设计了一个 undefined。区别是这样的：null 是一个表示“空”的对象，转为数值时为 0；undefined 是一个表示“此处尚未定义”的值，转为数值时为 NaN

null和undefined

- null表示“此处不应该有值” (intentional absence of value) ，典型用法如下：
 - 作为函数的参数，表示该函数的参数为空
 - 作为对象原型链的终点

null和undefined

- **undefined**表示“缺少值”，就是此处应该有一个值，但是还没有定义，典型用法如下：

```
// 变量声明了，但没有赋值
```

```
var i;
```

```
i // undefined
```

```
// 调用函数时，应该提供的参数没有提供，该参数等于 undefined
```

```
function f(x) {
```

```
    return x;
```

```
}
```

```
f() // undefined
```

```
// 对象没有赋值的属性
```

```
var o = new Object();
```

```
o.p // undefined
```

```
// 函数没有返回值时，默认返回 undefined
```

```
function f() {}
```

```
f() // undefined
```

null和undefined



<https://stackoverflow.com/questions/801032/why-is-null-an-object-and-whats-the-difference-between-null-and-undefined>

形象回答

```
(name is undefined)
```

You: What is `name` ? (*)

JavaScript: `name` ? What's a `name` ? I don't know what you're talking about. You haven't ever mentioned any `name` before. Are you seeing some other scripting language on the (client-)side?

```
name = null;
```

You: What is `name` ?

JavaScript: I don't know.

In short; **undefined** is where no notion of the thing exists; it has no type, and it's never been referenced before in that scope; **null** is where the thing is known to exist, but it's not known what the value is.

null和undefined

- null 与 undefined 的不同点:

```
Number(null) // 0  
5 + null // 5
```

```
Number(undefined) // NaN  
5 + undefined // NaN
```

```
1  typeof null      // "object" (因为一些以前的原因而不是'null')  
2  typeof undefined // "undefined"  
3  null === undefined // false  
4  null == undefined // true  
5  null === null // true  
6  null == null // true  
7  !null //true  
8  isNaN(1 + null) // false  
9  isNaN(1 + undefined) // true
```

undefined vs null in JavaScript

<https://howtodoinjava.com/typescript/undefined-vs-null/>

```
var myUndefinedVar;           //undefined  
var myNullVar = null;         //null  
  
myUndefinedVar + 100;         //NaN  
  
myNullVar + 100;              //100
```

主要内容

- 基本语法
- 数据类型
 - null和undefined
 - 布尔值
 - 数值
 - 字符串
 - 对象
- 运算符
- 标准库

布尔值

- 布尔值代表“真”和“假”两个状态，“真”用关键字`true`表示，“假”用关键字`false`表示，布尔值只有这两个值
- 下列运算符会返回布尔值：
 - 前置逻辑运算符：`!(Not)`
 - 相等运算符：`===`，`!==`，`==`，`!=`
 - 比较运算符：`>`，`>=`，`<`，`<=`

布尔值

- 如果 JavaScript 预期某个位置应该是布尔值，会将该位置上现有的值自动转为布尔值。转换规则是除了下面六个值被转为false，其他值都视为true
 - undefined
 - null
 - false
 - 0
 - NaN
 - ""或"（空字符串）

布尔值

```
if (') {  
  console.log('true');  
}  
// 没有任何输出
```

if命令后面的判断条件，预期应该是一个布尔值，所以JavaScript 自动将空字符串，转为布尔值false，导致程序不会进入代码块，所以没有任何输出

```
if ([]) {  
  console.log('true');  
}  
// true  
  
if ({}) {  
  console.log('true');  
}  
// true
```

注意，空数组（[]）和空对象（{}）对应的布尔值，都是true

```
>> Boolean([])  
← true  
>> Boolean({})  
← true
```

主要内容

- 基本语法
- 数据类型
 - null和undefined
 - 布尔值
 - 数值
 - 字符串
 - 对象
- 运算符
- 标准库

数值

- 根据 ECMAScript 标准，JavaScript 中只有一种数字类型：基于 IEEE 754 标准的双精度 64 位二进制格式的值 ($-(2^{63}-1)$ 到 $2^{63}-1$)
- JavaScript 中整数也是以 64 位浮点数形式储存，
- 除了浮点数外，JavaScript 中还有一些带符号的值：
 - $+\text{Infinity}$ ， $-\text{Infinity}$ 和 NaN (非数值，Not-a-Number)

数值

- JavaScript 提供 Number 对象的 MAX_VALUE 和 MIN_VALUE 属性，返回可以表示的最大值和最小值

```
Number.MAX_VALUE // 1.7976931348623157e+308  
Number.MIN_VALUE // 5e-324
```

- 在 ECMAScript 6 中，提供方法检查值是否在双精度浮点数的取值范围内，超出这个范围的数字不再安全
 - Number.isSafeInteger()
 - Number.MAX_SAFE_INTEGER
 - Number.MIN_SAFE_INTEGER

最小正数: $5e-324$, 即 5×10^{-324}

最大正数: $1.7976931348623157e+308$, 即 $1.7976931348623157 \times 10^{308}$

负的最小负数 (即负的最接近零的数): $-5e-324$

负的最大负数 (即负的最大正数): $-1.7976931348623157e+308$

https://zhuanlan.zhihu.com/p/410729303?utm_id=0

```
console.log(Number.isSafeInteger(42)); // true
```

```
console.log(Number.isSafeInteger(9007199254740992)); // false, 超出安全整数范围
```

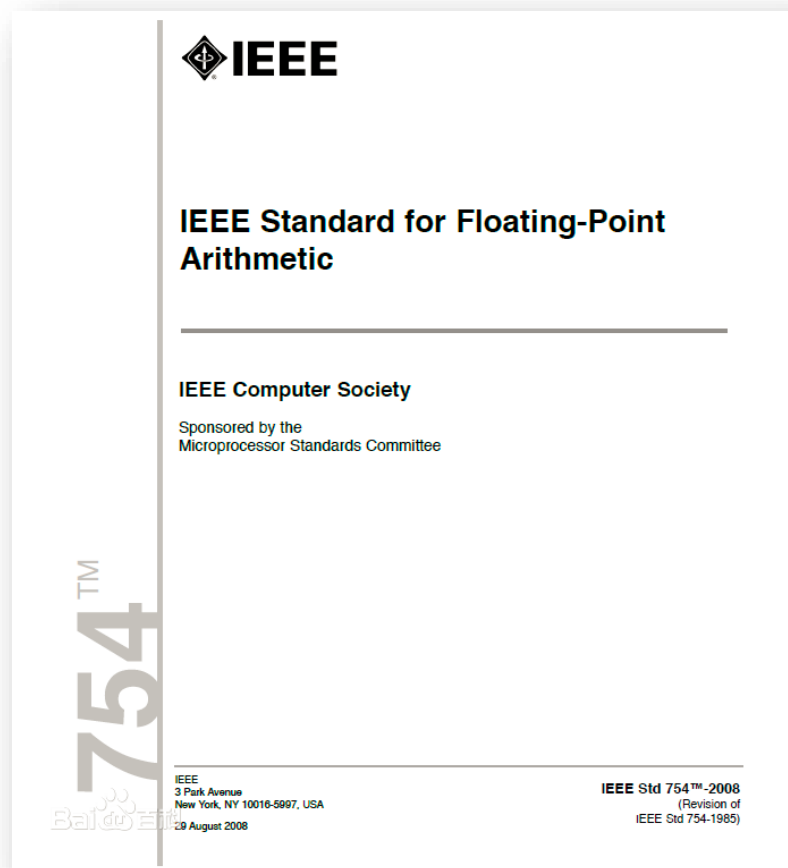
数值

- 在JavaScript中，绝对值小于 2^{53} 的整数，即 $-2^{53} + 1$ 到 $2^{53} - 1$ ，都可以精确表示

```
> Number.isSafeInteger(Math.pow(2, 53))  
< false  
  
> Number.isSafeInteger(-1*Math.pow(2, 53))  
< false  
  
> Number.isSafeInteger(Math.pow(2, 53)-1)  
< true  
  
> Number.isSafeInteger(-1*Math.pow(2, 53)+1)  
< true  
>
```

扩展阅读：What Every JavaScript Developer Should Know About Floating Points

<https://modernweb.com/what-every-javascript-developer-should-know-about-floating-points/>

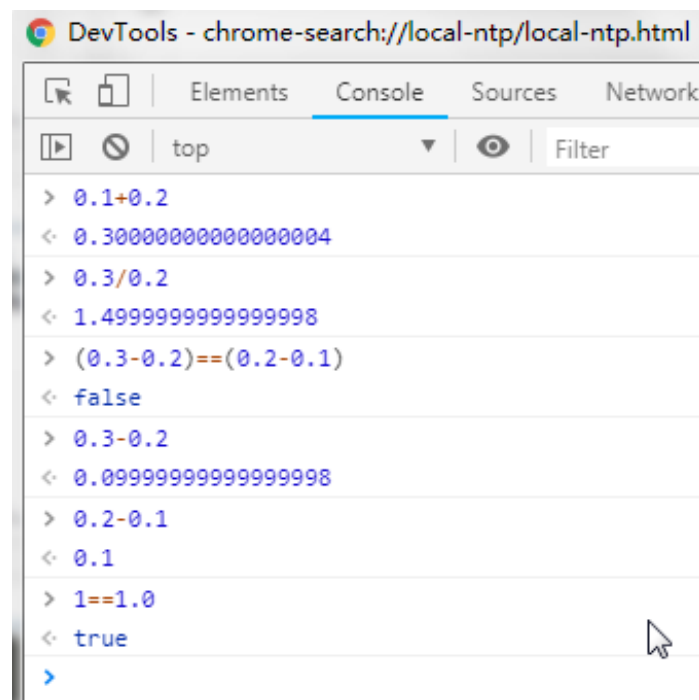


数值

- 由于浮点数不是精确的值，所以涉及小数的比较和运算要特别小心

在计算机中，浮点数采用二进制来表示，并且使用有限的位数来存储它们，因此浮点数无法精确地表示所有实数。

因为 0.1 和 0.2 这样的十进制小数无法完全准确地表示为二进制浮点数。在二进制中，只能使用有限的位数表示分数，而这些分数可能无法精确等于 0.1、0.2 或 0.3。



The screenshot shows the Chrome DevTools Console with the following code and results:

```
> 0.1+0.2
< 0.30000000000000004

> 0.3/0.2
< 1.4999999999999998

> (0.3-0.2)==(0.2-0.1)
< false

> 0.3-0.2
< 0.09999999999999998

> 0.2-0.1
< 0.1

> 1==1.0
< true
```

数值

- 数值的表示法

- JavaScript 的数值有多种表示方法，可以用字面形式直接表示，比如35（十进制）和0xFF（十六进制）
- 科学计数法

```
123e3 // 123000
```

```
123e-3 // 0.123
```

```
-3.1E+12
```

```
.1e-23
```

数值

- 以下两种情况，JavaScript 会自动将数值转为科学计数法表示，其他情况都采用字面形式直接表示：

```
1234567890123456789012  
// 1.2345678901234568e+21
```

小数点前的数字多于21位

```
123456789012345678901  
// 123456789012345680000
```

```
// 小数点后紧跟5个以上的零，  
// 就自动转为科学计数法  
0.000003 // 3e-7
```

小数点后的零多于5个

```
// 否则，就保持原来的字面形式  
0.000003 // 0.000003
```

数值

- JavaScript 对整数提供四种进制的表示方法：十进制、十六进制、八进制、二进制。默认情况下，JavaScript 内部会自动将八进制、十六进制、二进制转为十进制
 - 十进制：没有前导0的数值。
 - 八进制：有前缀 `0o` 或 `00` 的数值，或者有前导0、且只用到0-7的八个阿拉伯数字的数值。
 - 十六进制：有前缀 `0x` 或 `0X` 的数值。
 - 二进制：有前缀 `0b` 或 `0B` 的数值。

```
0xff // 255
0o377 // 255
0b11 // 3
```

有前导0的数值会被视为八进制，但是如果前导0后面有数字8和9，则该数值被视为十进制。前导0表示八进制，处理时很容易造成混乱。ES5 的严格模式和 ES6，已经废除了这种表示法，但是浏览器为了兼容以前的代码，目前还继续支持这种表示法

数值

- 特殊数值 - 正零和负零
 - JavaScript 的64位浮点数之中，有一个二进制位是符号位。这意味着，任何一个数都有一个对应的负值，就连0也不例外
 - JavaScript 内部实际上存在2个0：一个是+0，一个是-0，区别就是64位浮点数表示法的符号位不同。它们是等价的

```
-0 === +0 // true  
0 === -0 // true  
0 === +0 // true
```

数值

- 特殊数值 - 正零和负零

- 几乎场合，正零和负零都会被当作正常的0，唯一有区别的情况是，+0或-0当作分母，返回的值是不相等的

```
+0 // 0  
-0 // 0  
(-0).toString() // '0'  
(+0).toString() // '0'
```

```
(1 / +0) === (1 / -0) // false
```

```
> 1/+0  
< Infinity  
-----  
> 1/-0  
< -Infinity
```

数值

- **NaN**是JavaScript的特殊值，表示“非数字”
(Not a Number)

```
> 5 - 'x' // NaN
< NaN
> Math.sqrt(-1) // NaN
< NaN
> 0 / 0 // NaN
< NaN
> typeof NaN // 'number'
< "number"
> NaN === NaN // false
< false
> [NaN].indexOf(NaN) // -1
< -1
> Boolean(NaN) // false
< false
> NaN + 32 // NaN
< NaN
> NaN+NaN
< NaN
```

代码运行时，会自动将字符串x转为数值，但是由于x不是数值，所以最后得到结果为NaN，表示它是“非数字” (NaN)

数值

- **NaN**是 JavaScript 的特殊值，表示“非数字”
(Not a Number)

```
> 5 - 'x' // NaN
< NaN
> Math.sqrt(-1) // NaN
< NaN
> 0 / 0 // NaN
< NaN
> typeof NaN // 'number'
< "number"
> NaN === NaN // false
< false
> [NaN].indexOf(NaN) // -1
< -1
> Boolean(NaN) // false
< false
> NaN + 32 // NaN
< NaN
> NaN+NaN
< NaN
```

当发生错误时，数学函数的运算结果会出现NaN

数值

- **NaN**是JavaScript 的特殊值，表示“非数字”
(Not a Number)

```
> 5 - 'x' // NaN
< NaN
> Math.sqrt(-1) // NaN
< NaN
> 0 / 0 // NaN
< NaN
> typeof NaN // 'number'
< "number"
> NaN === NaN // false
< false
> [NaN].indexOf(NaN) // -1
< -1
> Boolean(NaN) // false
< false
> NaN + 32 // NaN
< NaN
> NaN+NaN
< NaN
```

NaN不是独立的数据类型，而是一个特殊数值，它的数据类型依然属于Number

数值

- **NaN**是 JavaScript 的特殊值，表示“非数字”
(Not a Number)

```
> 5 - 'x' // NaN
< NaN
> Math.sqrt(-1) // NaN
< NaN
> 0 / 0 // NaN
< NaN
> typeof NaN // 'number'
< "number"
> NaN === NaN // false
< false
> [NaN].indexOf(NaN) // -1
< -1
> Boolean(NaN) // false
< false
> NaN + 32 // NaN
< NaN
> NaN+NaN
< NaN
```

NaN不等于任何值，包括它本身

数值

- **NaN**是JavaScript的特殊值，表示“非数字”
(Not a Number)

```
> 5 - 'x' // NaN
< NaN
> Math.sqrt(-1) // NaN
< NaN
> 0 / 0 // NaN
< NaN
> typeof NaN // 'number'
< "number"
> NaN === NaN // false
< false
> [NaN].indexOf(NaN) // -1
< -1
> Boolean(NaN) // false
< false
> NaN + 32 // NaN
< NaN
> NaN+NaN
< NaN
```

NaN在布尔运算时被当作false
NaN与任何数（包括它自己）的运算，得到的都是NaN

越南方向的黑客组织

针对汽车企业的APT攻击

越南海水牛组织

- 2011年，越南总参信息技术部成立，随后不到一年时间，外界首次观察到了越南APT组织海水牛的网络活动
- 2018年1月，越南网络空间作战司令部（Command 86）成立，主要负责网络空间作战，并直接向中央军委汇报工作。同年4月，海水牛组织的网络作战任务随之更新
- 基于上述时间重叠情况推测，海水牛组织较大概率具有越南军方背景

越南汽车业发展概况

